

## Урок 29. Корутины. Практика. Работа с View

18 декабря 2020

В этом уроке создаем suspend функции и Flow из колбэков View компонентов

В прошлых примерах мы использовали Flow и suspend функции для работы с сетью, диском, вычислениями и прочими тяжелыми операциями. Но их вполне можно применять и при работе с UI. В основном это касается работы с различными listener для View компонентов.

Давайте рассмотрим несколько сценариев.

### Listener to suspend function

View listener может быть конвертирован в suspend функцию. Это даст лучшую читаемость кода.

Рассмотрим простой пример:

```
1 btn.text = "New text"
2 log("new width is ${btn.width}")
```

Мы меняем текст у кнопки и сразу после этого хотим узнать ее новую ширину.

Этот код не сработает как ожидалось. Мы получим старую ширину кнопки. Потому что setText не выполнится сразу в момент вызова кода. Реальное обновление кнопки будет помещено в очередь и произойдет чуть позже.

У нас есть способ узнать, когда именно это произойдет. Для этого мы можем использовать OnLayoutChangeListener:

```
1 val listener = object : View.OnLayoutChangeListener {
2     override fun onLayoutChange(view: View?, left: Int, top: Int, right: Int, bottom: Int, oldLeft: Int, oldTop: Int, ol
3         view?.removeOnLayoutChangeListener(this)
4         log("new width is ${btn.width}")
5     }
6 }
7 btn.addOnLayoutChangeListener(listener)
8
9 btn.text = "New text"
10 log("old width is ${btn.width}")
```

Создаем listener и вешаем его на кнопку. Он сработает, когда произойдут реальные изменения layout кнопки. Внутри отписываем listener и выводим в лог новую ширину.

В итоге этот код поместит обновления кнопки в очередь и выведет в лог старую ширину. Затем сработает listener после того, как кнопка будет обновлена и перерисована. И в нем мы уже будем видеть новую ширину кнопки.

Этот OnLayoutChangeListener может быть упакован в отдельную suspend функцию:

```
1 suspend fun View.awaitLayoutChange() = suspendCancellableCoroutine<Unit> { cont ->
2
3     val listener = object : View.OnLayoutChangeListener {
4
5         override fun onLayoutChange(view: View?, left: Int, top: Int,
6             right: Int, bottom: Int, oldLeft: Int, oldTop: Int, oldRight: Int, oldBottom: Int) {
7             view?.removeOnLayoutChangeListener(this)
8             cont.resume(Unit)
9         }
10    }
11
12    addOnLayoutChangeListener(listener)
13
14    cont.invokeOnCancellation { removeOnLayoutChangeListener(listener) }
```

Используем suspendCancellableCoroutine для создания отменяемой suspend функции. Создаем listener и вешаем его на View, для которого будет вызвана эта suspend функция. В onLayoutChange отписываем listener от View и сообщаем корутине, что можно продолжать работу.

invokeOnCancellation будет вызван при отмене корутины. В этом случае отписываем listener от View.

Теперь наш пример выглядит так:

```
1 lifecycleScope.launch {
2     btn.text = "New text"
3     log("old width is ${btn.width}")
4     btn.awaitLayoutChange()
5     log("new width is ${btn.width}")
6 }
```

Используем корутину, т.к. нам теперь необходимо вызывать suspend функцию awaitLayoutChange. Эта функция приостановит код корутины, пока кнопка не будет реально обновлена. После чего мы запрашиваем новую ширину кнопки.

Т.е. listener мы поместили в suspend функцию, и код стал читабельнее.

Аналогично можно обернуть listener анимации, чтобы можно было запускать анимацию и дожидаться ее окончания

```
1 lifecycleScope.launch {
2     val animator = ObjectAnimator.ofFloat(imageView, View.ALPHA, 0f, 1f)
3     animator.start()
4     animator.awaitEnd()
5 }
```

awaitEnd - suspend функция, которая приостановит корутину, пока анимация не завершится. Если у нас есть несколько анимаций, то мы сможем запускать их параллельно, последовательно, с задержкой, с повторами и т.д. В этом помогут корутины и подобные suspend функции. Более подробно об этом можно прочесть в [статье](#). Если вас интересуют более сложные примеры с анимацией и RecyclerView, то посмотрите эту [статью](#). Там используется та же схема, что и в простом примере, который мы рассмотрели.

## Listener to Flow

Если listener предполагает отправку нам нескольких значений, то мы можем сделать из него Flow.

Самый очевидный пример - это обработчик нажатия на кнопку:

```
1 val flow = callbackFlow {
2     btn.setOnClickListener {
3         offer(Unit)
4     }
5     awaitClose { btn.setOnClickListener(null) }
6 }
```

Билдер callbackFlow похож на channelFlow, который мы разбирали в Уроке 21. Под капотом он также создает комбинацию Flow + корутина + канал. Его отличие от channelFlow в том, что он требует вызова awaitClose в конце, иначе выдаст ошибку. Позже я дополню Урок 21 и распишу там подробнее эту разницу.

Код внутри callbackFlow будет выполнен, когда получатель подпишется (collect) на этот созданный Flow. Listener будет создан и повешен на кнопку. Метод awaitClose приостановит код.

По нажатию на кнопку данные будут отправляться (offer) получателю. Когда получатель отпишется, то метод awaitClose возобновит работу и отвяжет listener от кнопки.

В результате мы получим Flow, который будет сообщать о нажатиях на кнопку.

Еще один распространенный пример, это отслеживание изменений в EditText:

```
1 val flow = callbackFlow<String> {
2
3     val textWatcher = object: TextWatcher {
4
5         override fun afterTextChanged(editable: Editable?) {
6             editable?.toString()?.let{ offer(it)}
7         }
8
9         override fun beforeTextChanged(s: CharSequence?, start: Int, count: Int, after: Int) {}
10        override fun onTextChanged(s: CharSequence?, start: Int, before: Int, count: Int) {}
11    }
12
13    editText.addTextChangedListener(textWatcher)
14    awaitClose { editText.removeTextChangedListener(textWatcher) }
15 }
```

Схема та же, просто другой listener, который мониторит EditText и отправляет новые значения получателю.

В результате у нас есть Flow, который шлет нам изменения текста. Нам остается только добавить нужные операторы: map, filter, debounce и прочее. Такой Flow может быть использован для удобной реализации поиска или фильтра.

Когда мы используем схему MVVM, то не очень понятно куда девать эти Flow, привязанные к View компонентам. Их нельзя передавать в модель. Потому что модель не хранит никакие ссылки на View. А эти Flow через listener будут хранить ссылки на компоненты View.

Есть еще одно решение, как listener можно обернуть в Flow.

## Listener to Flow in ViewModel

Чтобы не создавать Flow на стороне View, мы можем создать его в модели. Раньше для этих целей использовались каналы с оберткой asFlow. View передавал данные из listener сразу в модель, модель внутри себя помещала их в канал, а канал передавал их в Flow. В итоге получалась трансформация обычных данных в Flow.

С выходом SharedFlow и StateFlow схема упростилась. Рассмотрим на примере поиска.

В модели у нас есть StateFlow и метод search, которые помещает данные в StateFlow.

```
1 val _searchQuery = MutableStateFlow("")
2
3 fun search(query: String) {
4     _searchQuery.value = query
5 }
```

В View мы вешаем listener на EditText и в нем просто вызываем метод модели search и передаем туда данные.

```
1 | editText.addTextChangedListener { model.search(it.toString()) }
```

Метод `addTextChangedListener` - это Kotlin расширение из `androidx.core:core-ktx`. Под капотом у него используется все тот же `TextWatcher`.

В итоге в модели у нас есть `StateFlow`, который предоставляет данные из `EditText`.

Мы можем добавить к этому Flow операторов и попросить при каждом новом значении выполнять поиск, например, с помощью `UseCase`:

```
1 | val searchResultFlow = _searchQuery.asStateFlow()
2 |   .debounce(500)
3 |   .filter { it.length > 3 }
4 |   .mapLatest { query ->
5 |       searchUseCase.execute(query)
6 |   }
```

В итоге `searchResultFlow` будет возвращать нам готовый результат поиска. Его уже можно использовать, как нам требуется. Например, конвертировать в `LiveData`.